R Programming

Unit 3 - Assignment

Basic Grocery Store Analysis

Subject	R Programming
Unit	Unit 3 - Control Structures & Functions
Assignment	Basic Grocery Store Analysis
Name	Pawar Shubham Dilip
MIS No	752562026

Dataset Creation

Create the grocery store customer dataset as a data frame with the required columns.

```
# ── Grocery Store Customer Dataset ──
customer_data <- data.frame(
  CustomerID   = c(101, 102, 103, 104, 105, 106, 107, 108),
  AmountSpent  = c(2500, 1800, 500, 3200, 950, 1500, 2100, 800),
  Visits       = c(5, 3, 1, 7, 2, 4, 6, 2),
  Membership   = c("Gold", "Silver", "Bronze", "Gold",
                   "Bronze", "Silver", "Gold", "Bronze"))
# View the dataset
print(customer_data)
str(customer_data)
```

```
R 4.5.3 · ~/
> # ── Grocery Store Customer Dataset ──
> customer_data <- data.frame(
+   CustomerID   = c(101, 102, 103, 104, 105, 106, 107, 108),
+   AmountSpent  = c(2500, 1800, 500, 3200, 950, 1500, 2100, 800),
+   Visits       = c(5, 3, 1, 7, 2, 4, 6, 2),
+   Membership   = c("Gold", "Silver", "Bronze", "Gold",
+                    "Bronze", "Silver", "Gold", "Bronze")
+ )
>
> # View the dataset
> print(customer_data)
  CustomerID AmountSpent Visits Membership
1        101        2500      5       Gold
2        102        1800      3     Silver
3        103         500      1     Bronze
4        104        3200      7       Gold
5        105         950      2     Bronze
6        106        1500      4     Silver
7        107        2100      6       Gold
8        108         800      2     Bronze
> str(customer_data)
'data.frame':   8 obs. of  4 variables:
 $ CustomerID : num  101 102 103 104 105 106 107 108
 $ AmountSpent: num  2500 1800 500 3200 950 1500 2100 800
 $ Visits     : num  5 3 1 7 2 4 6 2
 $ Membership : chr  "Gold" "Silver" "Bronze" "Gold" ...
>
```

Task 1: Using Conditions - Category Column

Add a new column 'Category' based on the amount spent, using ifelse() for vectorised conditional logic.

```
# Task 1: Create Category column using nested ifelse()
# ifelse(test, yes, no) - vectorised, works on entire column at once
customer_data$Category <- ifelse(
  customer_data$AmountSpent > 2000, "High",
  ifelse(
    customer_data$AmountSpent >= 1000, "Medium",
    "Low"
  )
)

# Display updated dataset
print(customer_data[ , c("CustomerID", "AmountSpent", "Category")])
```

```
R 4.5.3 · ~/
> # Task 1: Create Category column using nested ifelse()
> # ifelse(test, yes, no) - vectorised, works on entire column at once
> customer_data$Category <- ifelse(
+   customer_data$AmountSpent > 2000, "High",
+   ifelse(
+     customer_data$AmountSpent >= 1000, "Medium",
+     "Low"
+   )
+ )
>
> # Display updated dataset
> print(customer_data[ , c("CustomerID", "AmountSpent", "Category")])
  CustomerID AmountSpent Category
1        101        2500     High
2        102        1800   Medium
3        103         500      Low
4        104        3200     High
5        105         950      Low
6        106        1500   Medium
7        107        2100     High
8        108         800      Low
>
```

Task 2: Using Loop

Use a for loop to iterate through each row and print the Customer ID along with their spending category.

```
# Task 2: Loop through each customer and print ID + Category
cat("\n=== Customer Category Report ===\n")
```

```
for (i in 1:nrow(customer_data)) {
  cat("Customer ID:", customer_data$CustomerID[i],
      "| Category:", customer_data$Category[i], "\n")
}
```

```
R 4.5.3 · ~/
> # Task 2: Loop through each customer and print ID + Category
> cat("\n=== Customer Category Report ===\n")

=== Customer Category Report ===
>
> for (i in 1:nrow(customer_data)) {
+   cat("Customer ID:", customer_data$CustomerID[i],
+       "| Category:", customer_data$Category[i], "\n")
+ }
Customer ID: 101 | Category: High
Customer ID: 102 | Category: Medium
Customer ID: 103 | Category: Low
Customer ID: 104 | Category: High
Customer ID: 105 | Category: Low
Customer ID: 106 | Category: Medium
Customer ID: 107 | Category: High
Customer ID: 108 | Category: Low
>
```

Task 3: Control Functions - next and break

Use next to skip customers with fewer than 3 visits, and break to stop execution if a negative amount is found.

```
# Task 3: Control flow with next (skip) and break (stop)
cat("\n=== Filtered Customer Report (Visits >= 3) ===\n")
for (i in 1:nrow(customer_data)) {

  # STOP: break if any amount is negative (data integrity check)
  if (customer_data$AmountSpent[i] < 0) {
    cat("ERROR: Negative amount found for Customer",
        customer_data$CustomerID[i], "- Stopping.\n")
    break
  }

  # SKIP: next if visits < 3 (not enough engagement)
  if (customer_data$Visits[i] < 3) {
    cat("Skipping Customer", customer_data$CustomerID[i],
        "(only", customer_data$Visits[i], "visit(s))\n")
    next
  }

  # Process eligible customers
  cat("Customer:", customer_data$CustomerID[i],
      "| Visits:", customer_data$Visits[i],
      "| Amount: Rs.", customer_data$AmountSpent[i], "\n")
}
```

```
R 4.5.3 · ~/
> # Task 3: Control flow with next (skip) and break (stop)
> cat("\n=== Filtered Customer Report (Visits >= 3) ===\n")

=== Filtered Customer Report (Visits >= 3) ===
>
> for (i in 1:nrow(customer_data)) {
+
+   # STOP: break if any amount is negative (data integrity check)
+   if (customer_data$AmountSpent[i] < 0) {
+     cat("ERROR: Negative amount found for Customer",
+         customer_data$CustomerID[i], "- Stopping.\n")
+     break
+   }
+
+   # SKIP: next if visits < 3 (not enough engagement)
+   if (customer_data$Visits[i] < 3) {
+     cat("Skipping Customer", customer_data$CustomerID[i],
+         "(only", customer_data$Visits[i], "visit(s))\n")
+     next
+   }
+
+   # Process eligible customers
+   cat("Customer:", customer_data$CustomerID[i],
+       "| Visits:", customer_data$Visits[i],
+       "| Amount: Rs.", customer_data$AmountSpent[i], "\n")
+ }
Customer: 101 | Visits: 5 | Amount: Rs. 2500
Customer: 102 | Visits: 3 | Amount: Rs. 1800
Skipping Customer 103 (only 1 visit(s))
Customer: 104 | Visits: 7 | Amount: Rs. 3200
Skipping Customer 105 (only 2 visit(s))
Customer: 106 | Visits: 4 | Amount: Rs. 1500
Customer: 107 | Visits: 6 | Amount: Rs. 2100
Skipping Customer 108 (only 2 visit(s))
>
```

Task 4: Apply Family Functions

Use sapply() to compute average and maximum amount spent from the dataset. The apply family is preferred over loops for cleaner, faster vector operations.

```
# Task 4: Apply family functions

# sapply() applies a function to a vector and returns a simplified result
# Here we use it on a single-column numeric vector

amount_vector <- customer_data$AmountSpent

# Average amount spent
avg_amount <- sapply(list(avg = amount_vector), mean)
cat("Average Amount Spent: Rs.", round(avg_amount, 2), "\n")

# Maximum amount spent
max_amount <- sapply(list(max = amount_vector), max)
cat("Maximum Amount Spent: Rs.", max_amount, "\n")
# Alternative: Using apply() on a matrix column
```

```
marks_matrix <- as.matrix(customer_data$AmountSpent)
cat("Mean via apply():", apply(marks_matrix, 2, mean), "\n")
cat("Max  via apply():", apply(marks_matrix, 2, max),  "\n")
```

R 4.5.3 · ~/

```
> # Task 4: Apply family functions
>
> # sapply() applies a function to a vector and returns a simplified result
> # Here we use it on a single-column numeric vector
>
> amount_vector <- customer_data$AmountSpent
>
> # Average amount spent
> avg_amount <- sapply(list(avg = amount_vector), mean)
> cat("Average Amount Spent: Rs.", round(avg_amount, 2), "\n")
Average Amount Spent: Rs. 1668.75
> # Output: Average Amount Spent: Rs. 1543.75
>
> # Maximum amount spent
> max_amount <- sapply(list(max = amount_vector), max)
> cat("Maximum Amount Spent: Rs.", max_amount, "\n")
Maximum Amount Spent: Rs. 3200
> # Output: Maximum Amount Spent: Rs. 3200
>
> # Alternative: Using apply() on a matrix column
> marks_matrix <- as.matrix(customer_data$AmountSpent)
> cat("Mean via apply():", apply(marks_matrix, 2, mean), "\n")
Mean via apply(): 1668.75
> cat("Max  via apply():", apply(marks_matrix, 2, max),  "\n")
Max  via apply(): 3200
>
```

Task 5: Functions with Default Argument

Create a discount calculation function with a default membership parameter set to 'Bronze'.

```
# Task 5: Function with default argument
calculate_discount <- function(amount, membership = "Bronze") {
  # Define discount rates per membership tier
  rate <- switch(membership,
    "Gold"   = 0.20,   # 20%
    "Silver" = 0.10,   # 10%
    "Bronze" = 0.05,   # 5% (also the default)
    stop("Unknown membership type: ", membership)
  )
  discount_amount <- amount * rate
  final_price     <- amount - discount_amount

  cat("Membership :", membership, "\n")
  cat("Amount     : Rs.", amount, "\n")
  cat("Discount(%): ", rate * 100, "%\n")
  cat("Discount   : Rs.", discount_amount, "\n")
  cat("Final Price: Rs.", final_price, "\n\n")

  return(final_price)
}

# Test with specific memberships
calculate_discount(2500, "Gold")    # 20% off
calculate_discount(1800, "Silver")  # 10% off
calculate_discount(500)             # Default: Bronze, 5% off

# Apply to entire dataset
customer_data$FinalPrice <- mapply(calculate_discount,
                                   customer_data$AmountSpent,
                                   customer_data$Membership)
```

R 4.5.3 · ~/

```
> # Task 5: Function with default argument
> calculate_discount <- function(amount, membership = "Bronze") {
+   # Define discount rates per membership tier
+   rate <- switch(membership,
+                  "Gold"   = 0.20,   # 20%
+                  "Silver" = 0.10,   # 10%
+                  "Bronze" = 0.05,   # 5% (also the default)
+                  stop("Unknown membership type: ", membership)
+   )
+   discount_amount <- amount * rate
+   final_price     <- amount - discount_amount
+
+   cat("Membership :", membership, "\n")
+   cat("Amount     : Rs.", amount, "\n")
+   cat("Discount(%): ", rate * 100, "%\n")
+   cat("Discount   : Rs.", discount_amount, "\n")
+   cat("Final Price: Rs.", final_price, "\n\n")
+
+   return(final_price)
+ }
>
> # Test with specific memberships
> calculate_discount(2500, "Gold")    # 20% off
Membership : Gold
Amount     : Rs. 2500
Discount(%):  20 %
Discount   : Rs. 500
Final Price: Rs. 2000

[1] 2000
> calculate_discount(1800, "Silver")  # 10% off
Membership : Silver
Amount     : Rs. 1800
Discount(%):  10 %
Discount   : Rs. 180
Final Price: Rs. 1620

[1] 1620
> calculate_discount(500)             # Default: Bronze, 5% off
Membership : Bronze
Amount     : Rs. 500
```

```
Discount(%):  5 %
Discount   : Rs. 25
Final Price: Rs. 475

[1] 475
>
> # Apply to entire dataset
> customer_data$FinalPrice <- mapply(calculate_discount,
+                                    customer_data$AmountSpent,
+                                    customer_data$Membership)
Membership : Gold
Amount     : Rs. 2500
Discount(%):  20 %
Discount   : Rs. 500
Final Price: Rs. 2000

Membership : Silver
Amount     : Rs. 1800
Discount(%):  10 %
Discount   : Rs. 180
Final Price: Rs. 1620

Membership : Bronze
Amount     : Rs. 500
Discount(%):  5 %
Discount   : Rs. 25
Final Price: Rs. 475

Membership : Gold
Amount     : Rs. 3200
Discount(%):  20 %
Discount   : Rs. 640
Final Price: Rs. 2560

Membership : Bronze
Amount     : Rs. 950
Discount(%):  5 %
Discount   : Rs. 47.5
Final Price: Rs. 902.5
```

```
Membership : Silver
Amount     : Rs. 1500
Discount(%):  10 %
Discount   : Rs. 150
Final Price: Rs. 1350

Membership : Gold
Amount     : Rs. 2100
Discount(%):  20 %
Discount   : Rs. 420
Final Price: Rs. 1680

Membership : Bronze
Amount     : Rs. 800
Discount(%):  5 %
Discount   : Rs. 40
Final Price: Rs. 760

>
```

Task 6: Lazy Evaluation

Demonstrate lazy evaluation in R - function arguments are not evaluated until they are actually needed inside the function body.

```
# Task 6: Lazy Evaluation
# R uses lazy evaluation: arguments are only computed when first accessed.
# If an argument is never used, it is never evaluated - no error occurs.

lazy_discount <- function(amount, membership, verbose = FALSE) {
  # 'verbose' parameter is lazy - only evaluated if needed

  rate <- switch(membership,
    "Gold" = 0.20, "Silver" = 0.10, "Bronze" = 0.05
  )

  # Discount is calculated ONLY when this line is reached
  # If we returned early before this, 'rate' would never be used
  discount <- amount * rate

  # 'verbose' argument is only evaluated here, not at function call time
  if (verbose) {
    cat("[Verbose] Rate applied:", rate * 100, "% on Rs.", amount, "\n")
  }

  return(amount - discount)
}

# Without verbose: discount logic runs, verbose block is skipped
result1 <- lazy_discount(2500, "Gold")
cat("Result (no verbose):", result1, "\n")

# With verbose: discount logic runs AND verbose block executes
result2 <- lazy_discount(2500, "Gold", verbose = TRUE)
cat("Result (verbose):", result2, "\n")
# [Verbose] Rate applied: 20 % on Rs. 2500

# Lazy evaluation demo: unused argument causes no error
f <- function(x, y) x + 1  # y is never used
f(5)  # Works fine - R never evaluates the missing y
```

```
R 4.5.3 · ~/

> # Task 6: Lazy Evaluation
> # R uses lazy evaluation: arguments are only computed when first accessed.
> # If an argument is never used, it is never evaluated - no error occurs.
>
> lazy_discount <- function(amount, membership, verbose = FALSE) {
+   # 'verbose' parameter is lazy - only evaluated if needed
+
+   rate <- switch(membership,
+                  "Gold" = 0.20, "Silver" = 0.10, "Bronze" = 0.05
+   )
+
+   # Discount is calculated ONLY when this line is reached
+   # If we returned early before this, 'rate' would never be used
+   discount <- amount * rate
+
+   # 'verbose' argument is only evaluated here, not at function call time
+   if (verbose) {
+     cat("[Verbose] Rate applied:", rate * 100, "% on Rs.", amount, "\n")
+   }
+
+   return(amount - discount)
+ }
>
> # Without verbose: discount logic runs, verbose block is skipped
> result1 <- lazy_discount(2500, "Gold")
> cat("Result (no verbose):", result1, "\n")
Result (no verbose): 2000
>
> # With verbose: discount logic runs AND verbose block executes
> result2 <- lazy_discount(2500, "Gold", verbose = TRUE)
[Verbose] Rate applied: 20 % on Rs. 2500
> cat("Result (verbose):", result2, "\n")
Result (verbose): 2000
> # [Verbose] Rate applied: 20 % on Rs. 2500
>
> # Lazy evaluation demo: unused argument causes no error
> f <- function(x, y) x + 1  # y is never used
> f(5)  # Works fine - R never evaluates the missing y
[1] 6
>
```

Task 7: Objects and Classes

Create a list representing one customer and assign it the class 'Customer', demonstrating R's S3 object system.

```
# Task 7: Objects and Classes (S3 system)

# Create a list for Customer 101
customer_obj <- list(
  id         = 101,
  name       = "Rahul Sharma",
  amount     = 2500,
  visits     = 5,
  membership = "Gold",
  category   = "High"
)
```

```r
# Assign a class to the list - now it is a 'Customer' object
class(customer_obj) <- "Customer"

# Check the class
cat("Object class:", class(customer_obj), "\n")
# Output: Object class: Customer

# Define a custom print method for the Customer class
print.Customer <- function(obj) {
  cat("========== Customer Profile ==========\n")
  cat("ID        :", obj$id,         "\n")
  cat("Name      :", obj$name,       "\n")
  cat("Membership:", obj$membership, "\n")
  cat("Amount    : Rs.", obj$amount, "\n")
  cat("Visits    :", obj$visits,     "\n")
  cat("Category  :", obj$category,  "\n")
  cat("======================================\n")
}

# Calling print() will now use our custom method
print(customer_obj)

# Check structure
str(customer_obj)
is.list(customer_obj)
inherits(customer_obj, "Customer")
```

R 4.5.3 · ~/

```
[1] 6
> # Task 7: Objects and Classes (S3 system)
>
> # Create a list for Customer 101
> customer_obj <- list(
+   id         = 101,
+   name       = "Rahul Sharma",
+   amount     = 2500,
+   visits     = 5,
+   membership = "Gold",
+   category   = "High"
+ )
>
> # Assign a class to the list - now it is a 'Customer' object
> class(customer_obj) <- "Customer"
>
> # Check the class
> cat("Object class:", class(customer_obj), "\n")
Object class: Customer
> # Output: Object class: Customer
>
> # Define a custom print method for the Customer class
> print.Customer <- function(obj) {
+   cat("========== Customer Profile ==========\n")
+   cat("ID        :", obj$id,         "\n")
+   cat("Name      :", obj$name,       "\n")
+   cat("Membership:", obj$membership, "\n")
+   cat("Amount    : Rs.", obj$amount, "\n")
+   cat("Visits    :", obj$visits,     "\n")
+   cat("Category  :", obj$category,  "\n")
+   cat("======================================\n")
+ }
>
> # Calling print() will now use our custom method
> print(customer_obj)
========== Customer Profile ==========
ID        : 101
Name      : Rahul Sharma
Membership: Gold
Amount    : Rs. 2500
Visits    : 5
Category  : High
======================================
>
> # Check structure
> str(customer_obj)
List of 6
 $ id        : num 101
 $ name      : chr "Rahul Sharma"
 $ amount    : num 2500
 $ visits    : num 5
 $ membership: chr "Gold"
 $ category  : chr "High"
 - attr(*, "class")= chr "Customer"
> is.list(customer_obj)
[1] TRUE
> inherits(customer_obj, "Customer")
[1] TRUE
>
```

Task 8: Debugging

Demonstrate a common R error, identify the bug, and show the corrected version with explanation.

```
# Task 8: Debugging

# ── BUGGY FUNCTION (contains an error) ──
calculate_total_buggy <- function(prices) {
  total <- sum(price)   # BUG: 'price' should be 'prices' (wrong variable name)
  return(total)
}

# Calling the buggy function produces an error:
# calculate_total_buggy(c(100, 200, 300))
# Error in calculate_total_buggy(c(100, 200, 300)) :
#   object 'price' not found

# ── DEBUGGING STEPS ──
# Step 1: Read the error message carefully - 'price' not found
# Step 2: Check the function body - parameter name is 'prices'
# Step 3: The bug is a typo: sum(price) should be sum(prices)

# ── CORRECTED FUNCTION ──
calculate_total_fixed <- function(prices) {
  total <- sum(prices)  # FIXED: correct variable name
  return(total)
}

# Test corrected function
result <- calculate_total_fixed(c(100, 200, 300))
cat("Total price: Rs.", result, "\n")

# ── Additional Debugging Tools ──
# traceback()   - shows call stack after an error
# debug(fn)     - step through function line by line
# browser()     - insert inside function to pause and inspect
# print() / cat() - add diagnostic output at key points
```

```
R 4.5.3 · ~/
> # Task 8: Debugging
>
> # —— BUGGY FUNCTION (contains an error) ——
> calculate_total_buggy <- function(prices) {
+   total <- sum(price)   # BUG: 'price' should be 'prices' (wrong variable name)
+   return(total)
+ }
>
> # Calling the buggy function produces an error:
> # calculate_total_buggy(c(100, 200, 300))
> # Error in calculate_total_buggy(c(100, 200, 300)) :
> #   object 'price' not found
>
> # —— DEBUGGING STEPS ——
> # Step 1: Read the error message carefully - 'price' not found
> # Step 2: Check the function body - parameter name is 'prices'
> # Step 3: The bug is a typo: sum(price) should be sum(prices)
>
> # —— CORRECTED FUNCTION ——
> calculate_total_fixed <- function(prices) {
+   total <- sum(prices)  # FIXED: correct variable name
+   return(total)
+ }
>
> # Test corrected function
> result <- calculate_total_fixed(c(100, 200, 300))
> cat("Total price: Rs.", result, "\n")
Total price: Rs. 600
>
> # —— Additional Debugging Tools ——
> # traceback()   - shows call stack after an error
> # debug(fn)     - step through function line by line
> # browser()     - insert inside function to pause and inspect
> # print() / cat() - add diagnostic output at key points
>
```

End of Assignment

R Programming - Unit 3 | Control Structures, Functions & OOP